

Introducción a Nimble como herramienta de simulación MCMC

Introducción a la inferencia bayesiana basada en simulación

Miguel Ángel Martínez Beneito
miguel.a.martinez@uv.es



¿Qué es Nimble?

- Nimble es una herramienta para la **inferencia** en **modelos jerárquicos** bayesianos.
- Una **alternativa amigable** (basada en lenguaje bugs) para usuarios anteriores de WinBUGS, OpenBUGS o JAGS.
- Supone una **extensión**, programable, **de bugs** en la que resulta posible la implementación de nuevas distribuciones y funciones.
- Un sistema **configurable** para simulación **MCMC**, con algoritmos avanzados ya implementados.
- Una librería para la inferencia en **métodos no necesariamente** basados en **MCMC** (filtros de partículas para sequential Monte Carlo o métodos bayesianos para máxima verosimilitud).

Nimble, uso básico para simulación MCMC

- Nimble puede ser utilizado de forma **muy similar** al uso que hemos hecho hasta ahora de JAGS.
- Su **uso** consiste, básicamente, en:
 - **programación** en lenguaje BUGS del modelo del que se pretende simular.
 - **llamada a Nimble** con el modelo, datos, valores iniciales ... correspondientes.
- Nimble utiliza un “**dialecto**” de bugs, con algunas **particularidades** (extensiones del lenguaje bugs original).
- Entre las principales **particularidades** de la programación de modelos en Nimble destacan:
 - Posibilidad de **parametrizaciones** múltiples de las **distribuciones**.
 - Uso de **álgebra lineal** para declaración de **componentes deterministas** en modelos.
 - **Versión extendida** de funciones, distribuciones y utilidades.

Posibilidad de parametrizaciones multiples de las distribuciones:

- La distribución **Normal** en JAGS se parametrizaba en función de la **precisión**, cuyo uso resulta **poco intuitivo**. Algo parecido pasa con la distribución Gamma, Beta ...
- Nimble admite **distintas parametrizaciones** de sus distribuciones para facilitar su uso.
- Resultan **equivalentes**:

$x \sim \text{dnorm}(0, \text{tau})$, $x \sim \text{dnorm}(0, \text{var}=1/\text{tau})$ o

$x \sim \text{dnorm}(0, \text{sd}=1/\text{sqrt}(\text{tau}))$

- Nimble tiene una **paramerización “canónica”**/interna de cada distribución (la que utiliza la *sd* en el caso de la normal) y en caso de que utilicemos **otra**, la **traduce** a su correspondiente expresión canónica.
- Las distribuciones **Gamma o Beta** admiten parametrizaciones en función de su **media y desviación típica**.

- Podemos encontrar una relación completa de las **parametrizaciones de las distribuciones** de Nimble en la Tabla 5.2 de su **manual** del usuario¹.

Uso de álgebra lineal para declaración de componentes deterministas en modelos:

- La declaración de modelos en lenguaje BUGS para softwares más antiguos (WinBUGS, OpenBUGS, JAGS) **no permitía apenas** expresiones vectoriales del tipo: `x[1:2] <- y[1:2] + z[1:2]` que R sí admite.
- Nimble **sí** permite ese tipo de expresiones para **componentes deterministas**. Así, se permiten expresiones como:

```
mu[1:N] <- beta0 + betaX * x[1:N] + betaW * W[1:N]
```

```
C[1:I, 1:J] <- A[1:I, 1:K] %*% B[1:K, 1:J]
```

- Estas expresiones, además de válidas son, en general, **más eficientes** que la declaración individual de cada componente.

¹Disponible en: <https://r-nimble.org/manuals/NimbleUserManual.pdf>

- El uso de **expresiones vectoriales** para la definición de **variables** está **sólo** permitido en caso de que las variables definidas sigan una distribución **multivariante**. Así, es válido:

$$\text{mvx}[1:I] \sim \text{dmnorm}(\text{mu}[1:I], \text{cov} = \text{mvCov}[1:I, 1:I])$$

pero no:

$$\text{x}[1:I] \sim \text{dnorm}(\text{mu}, \text{sd} = \text{sdx})$$

- Aunque hay alternativas para evitarlo, el **uso de índices con la dimensión** de vectores/matrices cada vez que nos referimos a ellos es **muy conveniente**.

Versión extendida de funciones, distribuciones y utilidades:

- Nimble dispone de un **conjunto de funciones mucho más extenso** que el resto de **alternativas** basadas en lenguaje BUGS².

²para una descripción completa ver Tablas 5.4, 5.5 y 5.6 del manual del usuario de Nimble

- **Distribuciones** no disponibles en JAGS pero sí **en Nimble**:
 - Distribuciones **impropias** `dflat()`, `dhalfflat()`.
 - Distribuciones para **matrices**, simétricas, definidas positivas `dlkj_corr_cholesky`, `dinvwish`.
 - Distribuciones **espaciales** `dcar_normal`, `dcar_proper`.
- **Funciones** no disponibles en JAGS pero sí **en Nimble**:
 - Nimble implementa la **mayoría de funciones básicas** de R que podamos querer utilizar y otras **no tan básicas**, como la función modificada de Bessel de segunda clase o los cuantiles, funciones de densidad y probabilidades acumuladas de sus distribuciones.
 - Nimble implementa un buen número de **funciones matriciales** de gran interés como (log-)determinantes, descomposición SVD, descomposición de Cholesky, resolución de sistemas lineales o productos matriciales.
- Además, cualquier **función/distribución** que echáramos en falta se puede **programar** de manera relativamente asequible, desde R.

Definición de modelos en Nimble

- La principal función para **definir modelos** en Nimble es `nimbleCode`.
- Su **sintaxis** sería simplemente `nimbleCode({sintaxis del modelo})`, que crea un **objeto** de R que contiene el **modelo correspondiente**.

Ejemplo

- Consideramos el **banco de datos** `cars` trabajado en la práctica anterior.
- El **modelo de regresión** lineal simple, para su ejecución en Nimble, podría plantearse como:

```
modelo <- nimbleCode({  
  for(i in 1:N){dist[i] ~ dnorm(mu[i], sd = sigma)}  
  mu[1:N] <- beta0 + beta1 * velocidad[1:N]  
  beta0 ~ dflat()  
  beta1 ~ dflat()  
  sigma ~ dhalfflat()  
})
```

Ejecución sencilla de modelos en Nimble

- Una vez hemos definido el modelo estadístico correspondiente, Nimble admite su **ejecución** de manera **similar a JAGS**.
- La función `nimbleMCMC` de Nimble admite la **ejecución sencilla** del modelo correspondiente.
- Sus principales **argumentos** son:
 - **code**: código del **modelo**. Habitualmente un objeto generado por `nimbleCode`.
 - **data**: (named) lista con los **datos** del modelo. En Nimble los datos serían sólo aquellos valores del modelo cuya distribución define la función de **verosimilitud**.
 - **constants**: (named) lista con el **resto de datos** del modelo (dimensiones de vectores, covariables, parámetros de distribuciones iniciales ...).
 - **inits**: **valores iniciales** de las cadenas. Recomendable utilizar una **función generadora**, como hacíamos en JAGS, que será llamada para cada cadena.

- **monitors**: vector de **variables a monitorizar**. Equivalente a `parameters.to.save` en JAGS.
- **thin, niter, nburnin, nchains**: con interpretación **equivalente** a la de JAGS.
- **setSeed**: **semilla**, para control de la reproducibilidad de los resultados.
- **samples, summary, WAIC**: variables lógicas que controlan si el objeto devuelto contendrá los **valores simulados, summaries y WAIC** (criterio de selección de modelos). Sus valores por defecto son TRUE, FALSE y FALSE, respectivamente.
- **nimbleMCMC** devuelve, básicamente, una **lista con componentes samples, summary, WAIC** en función de si se han pedido o no estos elementos en la llamada a la función.
- La salida de **nimbleMCMC** es convenientemente **explotada** por funciones de la **librería MCMCvis**, como **MCMCtrace** o **MCMCsummary**.
- El uso de **MCMCsummary** presenta una **ventaja sobre** la opción **summary = TRUE** de **nimbleMCMC**. La primera opción calcula **estadísticos de convergencia** mientras que la segunda no.

Ejemplo

- Vamos a ejecutar el modelo de **regresión lineal** correspondiente al estudio de la distancia de **frenado** en coches.
- Definimos los **componentes básicos** necesarios para ejecutar el modelo:

```
datos <- list(dist = cars$dist)
constants <- list(velocidad = cars$speed, N = dim(cars)[1])
inits <- function(){list(beta0 = rnorm(1, mean(cars$dist)),
                          beta1 = rnorm(1,0,0.1), sigma = runif(1))}
```

- **Llamamos** ahora a la función `nimbleMCMC`:

```
library(nimble)
resulCars <- nimbleMCMC(code = modelo, data = datos, inits = inits,
                        monitors = c("beta0", "beta1", "sigma"),
                        constants = constants, nchains = 3,
                        thin = 1, niter = 2000, nburnin = 1000,
                        summary = TRUE)
```

- Valoramos la **convergencia** de la simulación mediante el **paquete** `MCMCvis`.

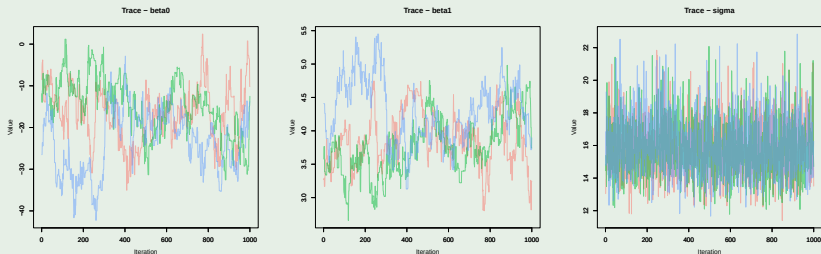
Ejemplo

```
round(MCMCvis::MCMCsummary(resulCars$samples),2)
```

```
##           mean    sd   2.5%   50% 97.5% Rhat n.eff
## beta0 -16.24  7.04 -28.76 -16.20 -2.92 1.34    47
## beta1   3.84  0.43   3.01   3.84  4.63 1.32    48
## sigma  15.86  1.71  13.00  15.71 19.71 1.01  2443
```

- El summary de la simulación es **bastante similar** al obtenido en JAGS.
- La **convergencia** no termina de ser **satisfactoria**.

```
MCMCvis::MCMCtrace(resulCars$samples, pdf = FALSE, type = "trace")
```



- La **convergencia** de β_0 y β_1 **presenta problemas**, ambas cadenas muestran correlación.

- Nimble parece mostrar (leves) **problemas de convergencia** que JAGS no mostraba, al fin y al cabo sus **algoritmos MCMC no son necesariamente los mismos** (Nimble $\rightarrow$ MH univariante, JAGS $\rightarrow$?).
- La **ejecución** de Nimble para este modelo es “**particularmente lenta**”, unos 30 segundos, bastante superior al de JAGS.
- Nimble **compila** específicamente el algoritmo MCMC correspondiente **en C++**, lo que puede llevarse cierto tiempo. En cualquier caso disponer del algoritmo compilado en C++ puede presentar **enormes ventajas computacionales** en modelos de ejecución más lenta.
- En cualquier caso, Nimble admite una **ejecución con mayor control** que la función `nimbleMCMC` que permite aliviar estos problemas.

Nimble, uso avanzado para simulación MCMC.

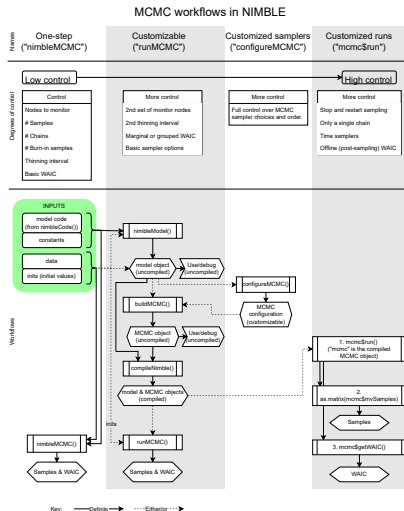


Figura 1: Flujo de trabajo de Nimble

- La ejecución de Nimble que hemos introducido se corresponde con la **columna izquierda** de la figura 1.
- Sin embargo, el **resto de columnas** describen un uso de Nimble que permite un **mayor control** del proceso y, por tanto, un uso **más adaptado** a nuestras necesidades.
- Nimble se asemeja más a un coche con **cambio manual**, a diferencia de otros softwares para MCMC, que se aproximarían más a **coches automáticos**.

- La **segunda columna** de la Figura 1 permite la **ejecución “paso a paso”** de un modelo Nimble.
- La ejecución **paso a paso** de Nimble requiere la **llamada consecutiva** a las funciones `nimbleModel`, `buildMCMC`, `compileNimble` y `runMCMC`.
 - `nimbleModel` **procesa el modelo**, datos, constantes e iniciales para generar un objeto de Nimble (`modelBaseClass`) con la información básica para poder simular del modelo.
 - `compileNimble` **compila** en C++ **el modelo** anterior.
 - `buildMCMC` **construye** el **algoritmo** de simulación **MCMC** correspondiente al modelo recién definido.
 - `compileNimble` **compila** en C++ **el algoritmo** anterior.
 - `runMCMC` **corre el algoritmo** compilado con la información del modelo que le hemos dado.
- `nimbleMCMC` básicamente reproduce la **llamada sucesiva de estas funciones** pero no permite una interacción tan directa con Nimble en cada paso de su ejecución.

Ejemplo

- El siguiente código permite la **ejecución paso a paso** de Nimble para el banco de datos cars, **equivalente** a la llamada anterior a nimbleMCMC.

```
nimModel <- nimbleModel(code = modelo, data = datos,  
                        constants = constants, inits = inits())  
compileNimble(nimModel)  
modBuilt <- buildMCMC(nimModel)  
modComp <- compileNimble(modBuilt, project = nimModel)  
resul <- runMCMC(modComp, nchains = 3, niter = 2000, thin = 1,  
                 nburnin = 1000, summary = TRUE)
```

```
round(MCMCvis::MCMCsummary(resul$samples),2)
```

##	mean	sd	2.5%	50%	97.5%	Rhat	n.eff
## beta0	-16.91	6.59	-29.44	-16.73	-3.09	1.15	48
## beta1	3.89	0.41	3.08	3.89	4.72	1.14	46
## sigma	15.75	1.69	12.90	15.61	19.43	1.01	2796

- De los casi **30 segundos de ejecución** del modelo sólo **0.1** corresponden a la función runMCMC que simula de la posterior.

- La llamada **paso a paso** de Nimble conlleva **ciertas ventajas** respecto al uso de nimbleMCMC:
 - Permite **resetear los datos**, no constantes, de un modelo y así correrlo de nuevo sin tener que recompilarlo.
 - Permite el **acceso a** algunas **opciones** de las funciones nimbleModel, buildMCMC ... que nimbleMCMC no permite.
 - Permite el **acceso a** las opciones descritas en la **tercera y cuarta columna** de la figura 1.
- La función configureMCMC permite **cambiar el algoritmo** MCMC propuesto por defecto por Nimble.
- Nimble dispone de un **grán número de algoritmos** de simulación para muestrear de cada uno de los parámetros de un modelo, como versiones multivariantes de Metropolis-Hastings, Slice sampling ...
- La librería nimbleHMC permite el uso de **métodos de simulación hamiltonianos** (mecanismo de simulación empleado en Stan), una versión avanzada de MCMC, con **convergencia** supuestamente muy **superior**.

- El uso de **métodos hamiltonianos** de simulación es en general **muy beneficioso**.
- El uso de métodos de simulación hamiltonianos **requiere**:
 - El uso de la **opción** `buildDerivs = TRUE` en la llamada a la función `nimbleModel`.
 - El uso de la **función** `configureHMC` previo a la construcción (`build`) del algoritmo MCMC correspondiente.

Ejemplo

- Vamos a simular de la distribución a posteriori del **modelo anterior**, pero empleando ahora **métodos** de simulación **hamiltonianos**.

```
library(nimbleHMC)
nimModel <- nimbleModel(code = modelo, data = datos,
                        constants = constants, inits = inits(),
                        buildDerivs = TRUE)
compileNimble(nimModel)
conf <- configureHMC(nimModel)
modBuilt <- buildMCMC(conf)
modComp <- compileNimble(modBuilt, project = nimModel)
```

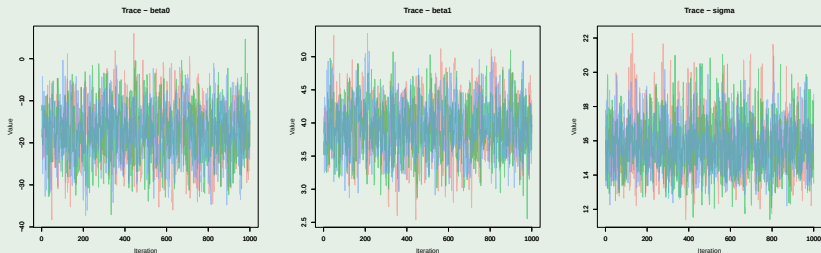
Ejemplo

```
resul <- runMCMC(modComp, nchains = 3, niter = 2000, thin = 1,  
                 nburnin = 1000, summary = TRUE)
```

```
round(MCMCvis::MCMCsummary(resul$samples),2)
```

##	mean	sd	2.5%	50%	97.5%	Rhat	n.eff
## beta0	-17.40	6.99	-30.48	-17.54	-2.83	1.01	962
## beta1	3.92	0.43	3.02	3.93	4.74	1.01	963
## sigma	15.77	1.66	12.95	15.63	19.53	1.00	1292

```
MCMCvis::MCMCtrace(resul$samples, pdf = FALSE, type = "trace")
```



- La ejecución de Nimble **paso a paso** permite beneficiarnos de lo descrito en la **cuarta columna** de la Figura 1.
- Básicamente, el **obviar** el uso de **la función** `runMCMC` permite:
 - **Retomar la simulación** en su estado final.
 - **Descarte de las iteraciones iniciales** que queramos, una vez haya terminado la simulación.
 - Cálculo de **summaries y WAIC** una vez damos la simulación por buena.
 - **Cambiar los parámetros** de la simulación, por ejemplo `thin`, conforme ésta avanza.
 - ...
- En resumen, permite el **control “en tiempo real”** del proceso de simulación.

Uso no tan avanzado, pero no tan friki, de Nimble

- El uso de Nimble plantea una **disyuntiva**:
 - ¿Usamos Nimble de manera **más cómoda**, pero con un **menor control** (perdiendo por ejemplo la posibilidad de utilizar métodos hamiltonianos)?
 - ¿Usamos Nimble **paso a paso**, pese a su **incómodo uso**?
- Aún cabe una **tercera opción**: Uso de alguna **función similar** a `nimbleMCMC`, que llame de manera secuencial al resto de funciones de Nimble, pero que permita un **mayor grado de personalización**.
- La **función** `pNimble` se plantea **ese objetivo**, junto con algún otro adicional.
- `pNimble` está **disponible en** Github, en el repositorio `pNimble` del usuario MigueBeneito, y puede ser cargado mediante el comando:
`source("https://raw.githubusercontent.com/MigueBeneito/pNimble/refs/heads/main/RutinasNimble.0.2.R")`.
- `pNimble` **pretende ser** en un futuro (cercano) parte de **una librería de R** de utilidades para Nimble.

- El uso adecuado de pNimble requiere la **instalación previa** de las librerías nimble, nimbleHMC, parallel, coda y ntfy.
- pNimble tiene **ventajas adicionales** al uso de nimbleMCMC, como:
 - Posibilidad de **ejecución en paralelo** de cada una de sus cadenas, acelerando la simulación por un factor de `n.chains` unidades.
 - Posibilidad de uso de **métodos** de simulación **hamiltonianos**.
 - Aviso por correo electrónico o **mensaje al móvil** del fin de la ejecución del modelo.
 - Posibilidad de **tunear la llamada** a Nimble con las opciones de las funciones de bajo nivel `nimbleModel`, `configureMCMC`, `buildMCMC` y `compileNimble`.
- La llamada básica a pNimble pretende ser **muy similar** a la de la **función** `nimbleMCMC`, de hecho los argumentos `code`, `data`, `constants`, `inits`, `thin`, `niter`, `nburnin`, `nchains`, `monitors`, `summary`, `WAIC` y tienen idéntica función en ambas.

- pNimble dispone además de los **argumentos adicionales**:
 - **parallel**: Con su valor por defecto, TRUE, **ejecuta** cada una de las **cadenas en paralelo** en un núcleo distinto. pNimble **reconstruye** la estructura **del objeto devuelto** como si la simulación no se hubiera llevado a cabo en paralelo. El uso de la opción `parallel = FALSE` es **interesante en caso de error**, ya que devuelve más información que la ejecución en paralelo.
 - **seeds**: **Semilla** para cada una de las cadenas, por defecto `1:n.chains`, para control de la reproducibilidad.
 - **HMC**: En caso de tomar el valor TRUE lleva a cabo un **muestreo** de tipo **hamiltoniano**. Su valor por defecto es FALSE.
 - **ntfyAccount**: **Cuenta** de la APP ntfy a la que enviar un **mensaje** cuando Nimble concluya su simulación. Muy útil para simulaciones largas (horas/días).
 - **email**: Similar a ntfyAccount pero con un **correo electrónico**, tiene un límite de 5 avisos diarios.
 - **control.model**, **control.compile**, **control.configure**, **control.build**: Listas con los **argumentos** a pasar a las llamadas a las **correspondientes funciones** dentro de pNimble.

- pNimble permite, por ahora, un grado de **control de las columnas 2 y 3** del flujo de trabajo de Nimble descrito en la Figura 1. En el **futuro** se espera extender dicho **control** también a su **cuarta columna**.

Ejemplo

- **Retomamos** la ejecución en Nimble del análisis del banco de datos cars.
- La ejecución **en paralelo** en Nimble, con muestreo **hamiltoniano y aviso** de final de ejecución, con la función pNimble sería tan sencilla como:

```
source("https://raw.githubusercontent.com/MigueBeneito/pNimble/refs/
resulCars <- pNimble(code = modelo, data = datos, inits = inits,
                    monitors = c("beta0", "beta1", "sigma"),
                    constants = constants, nchains = 3, thin = 1,
                    niter = 2000, nburnin = 1000, summary = TRUE,
                    HMC = TRUE, parallel = TRUE,
                    ntifyAccount = "MigueBeneito")
```

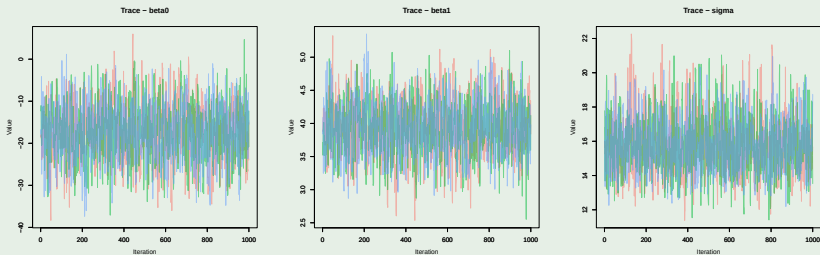
- A partir de aquí resulCars puede ser tratado **como cualquier otro objeto** de Nimble.

Ejemplo

```
round(MCMCvis::MCMCsummary(resulCars$samples),2)
```

```
##           mean    sd   2.5%   50% 97.5% Rhat n.eff
## beta0 -17.55  6.88 -30.59 -17.48 -3.65    1   996
## beta1   3.93  0.42   3.05   3.93  4.75    1  1192
## sigma  15.81  1.74  12.82  15.67 19.75    1  1321
```

```
MCMCvis::MCMCtrace(resulCars$samples, pdf=FALSE, type = "trace")
```



- La **convergencia ha mejorado** claramente con el uso de métodos **hamiltonianos**, además el cálculo en paralelo también ha **disminuido el tiempo** de computación.